Experiment 1 ? Program 4

DATA SCIENCE - 22DS102006

Data Wrangling

Perform hierarchical indexing by creating a series with a list of lists (or arrays) as the index, select subsets of data at outer and inner levels using partial indexing.

Rearrange the tabular data with hierarchical indexing using unstack and stack method.

Create two different DataFrames and merge them using index as merge key and combine data with overlap using combine_first method.

A). Perform hierarchical indexing by creating a series with a list of lists (or arrays) as the index, select subsets of data at outer and inner levels using partial indexing.

Aim

To create a Pandas Series with hierarchical (multi-level) indexing using a list of lists and to select subsets of data using partial indexing at the outer and inner levels.

Syntax

pd.MultiIndex.from_arrays(arrays, names=None)

To create a Series:

pd.Series(data, index=multi_index)

To select data:

series.loc['Outer_Level']

series.loc[('Outer_Level', 'Inner_Level')]

Solution Program

```
import pandas as pd
```

```
# Create a list of lists for hierarchical index
index = [
    ['Engineering', 'Engineering', 'Science', 'Science'],
    ['CSE', 'ECE', 'Physics', 'Chemistry']
]
# Create MultiIndex
multi_index = pd.MultiIndex.from_arrays(
    index,
    names=['Department', 'Branch']
)
# Create a Series using the hierarchical index
marks = pd.Series(
    [85, 78, 92, 88],
    index=multi_index
)
print("Original Series:")
print(marks)
# Partial indexing at the outer level
print("\nData for Engineering:")
print(marks.loc['Engineering'])
# Partial indexing at the inner level
print("\nData for CSE:")
print(marks.loc[('Engineering', 'CSE')])
# Selecting a subset using both levels
print("\nData for Science:")
print(marks.loc['Science'])
```

Output

```
Original Series:
Department   Branch
Engineering  CSE       85
             ECE       78
Science      Physics   92
             Chemistry 88
dtype: int64
Data for Engineering:
Branch
CSE    85
ECE    78
dtype: int64
Data for CSE:
85
Data for Science:
Branch
Physics      92
Chemistry    88
dtype: int64
```

Explanation

MultiIndex.from_arrays() creates a hierarchical index with two levels:

Outer level: Department

Inner level: Branch

marks.loc['Engineering'] selects all records under the outer level Engineering.

marks.loc[('Engineering', 'CSE')] selects a specific value using both outer and inner levels.

marks.loc['Science'] selects all records belonging to the Science department.

Result: The program successfully demonstrates hierarchical indexing and partial indexing at both outer and inner levels in Pandas.

B). Rearrange the tabular data with hierarchical indexing using unstack and stack method.

Aim

To rearrange tabular data with hierarchical indexing using the Pandas unstack() and stack() methods.

Syntax

unstack()

Converts one level of a hierarchical row index into columns.

Syntax

unstack()

Converts one level of a hierarchical row index into columns.

```
DataFrame.unstack(level=-1)
```

or

```
Series.unstack(level=-1)
```

stack()

Converts columns into a hierarchical row index.

```
DataFrame.stack()
```

or

```
Series.stack()
```

4. Dat Solution Program

```
import pandas as pd
```

```
# Create hierarchical index
index = pd.MultiIndex.from_tuples(
    [
        ('Engineering', 'CSE'),
        ('Engineering', 'ECE'),
        ('Science', 'Physics'),
        ('Science', 'Chemistry')
    ],
    names=['Department', 'Branch']
)
# Create tabular data
data = pd.DataFrame(
    {
        '2025': [85, 78, 92, 88],
        '2026': [90, 82, 95, 91]
    },
    index=index
)
print("Original Tabular Data:")
print(data)
# Unstack the inner level
unstacked_data = data.unstack()
print("\nData after Unstack():")
print(unstacked_data)
# Stack the data back
stacked_data = unstacked_data.stack()
```

```
print("\nData after Stack():")
```

```
print(stacked_data)
```

Output

```
Original Tabular Data:
```

```
                      2025  2026
```

```
Department   Branch
```

```
Engineering  CSE          85    90
```

```
             ECE          78    82
```

```
Science      Physics      92    95
```

```
             Chemistry    88    91
```

```
Data after Unstack():
```

```
             2025                   2026
```

```
Branch        CSE ECE Chemistry Physics CSE ECE Chemistry Physics
```

```
Department
```

```
Engineering     85  78       NaN     NaN  90  82       NaN     NaN
```

```
Science        NaN NaN      88.0    92.0 NaN NaN      91.0    95.0
```

```
Data after Stack():
```

```
Department   Branch
```

```
Engineering  CSE          85    90
```

```
             ECE          78    82
```

```
Science      Chemistry    88    91
```

```
             Physics      92    95
```

```
dtype: int64
```

Explanation

Hierarchical Indexing:
Department and Branch form the two levels of the hierarchical index.

unstack() method:

```
data.unstack()
```

Moves the inner index (Branch) from the rows to the columns.

stack() method:

```
unstacked_data.stack()
```

Moves the column labels back into the hierarchical row index.

Result

The unstack() method converts row-level hierarchical indexes into columns, while the stack() method converts columns back into hierarchical row indexes.

Aim

To create two different Pandas DataFrames, merge them using the index as the merge key, and combine their data using the combine_first() method to fill missing values with available values from another DataFrame.

Syntax

Merge using index

```
pd.merge(df1, df2, left_index=True, right_index=True)
```

combine_first()

```
df1.combine_first(df2)
```

The combine_first() method fills the missing (NaN) values in the first DataFrame with corresponding values from the second DataFrame.

Solution Program

```
import pandas as pd
```

```
# Create the first DataFrame
```

```
df1 = pd.DataFrame(
```

```
    {
```

```
        'Name': ['Ravi', 'Sita', 'Arun'],
```

```
        'Marks': [85, None, 78],
```

```
        'Grade': ['A', 'B', None]
```

```
    },
```

```
    index=[101, 102, 103]
)
# Create the second DataFrame
df2 = pd.DataFrame(
    {
        'Name': ['Ravi', 'Sita', 'Kiran'],
        'Marks': [90, 88, 82],
        'Grade': [None, 'A', 'B']
    },
    index=[101, 102, 104]
)
print("First DataFrame:")
print(df1)
print("\nSecond DataFrame:")
print(df2)
# Merge DataFrames using index as merge key
merged = pd.merge(
    df1,
    df2,
    left_index=True,
    right_index=True,
    how='outer',
    suffixes=('_DF1', '_DF2')
)
print("\nMerged DataFrame:")
print(merged)
```

```
# Combine data using combine_first()

combined = df1.combine_first(df2)

print("\nDataFrame after combine_first():")

print(combined)
```

Output

```
First DataFrame:

      Name  Marks Grade

101   Ravi   85.0     A

102   Sita    NaN     B

103   Arun   78.0  None

Second DataFrame:

      Name  Marks Grade

101   Ravi   90.0  None

102   Sita   88.0     A

104  Kiran   82.0     B

Merged DataFrame:

      Name_DF1  Marks_DF1 Grade_DF1 Name_DF2  Marks_DF2 Grade_DF2

101       Ravi       85.0         A     Ravi       90.0      None

102       Sita        NaN         B     Sita       88.0         A

103       Arun       78.0      None      NaN        NaN       NaN

104        NaN        NaN       NaN    Kiran       82.0         B

DataFrame after combine_first():

      Name  Marks Grade

101   Ravi   85.0     A

102   Sita   88.0     B

103   Arun   78.0  None
```

```
104  Kiran   82.0     B
```

Explanation

1. Creating DataFrames

Two DataFrames, df1 and df2, are created with student information. Their index values act as student IDs.

2. Merging using index

```
pd.merge(df1, df2, left_index=True, right_index=True, how='outer')
```

Here, the index is used as the merge key. The outer merge keeps all index values from both DataFrames.

3. Combining overlapping data

```
df1.combine_first(df2)
```

combine_first() uses values from df1 wherever they exist. If df1 contains a missing value, it takes the corresponding value from df2.

For example:

```
df1 Marks for 102 = NaN
```

```
df2 Marks for 102 = 88
```

After combine_first():

```
Marks for 102 = 88
```

Result

The program demonstrates both index-based merging and combining overlapping DataFrame data using **combine_first()**.

4. Data Wrangling

a. Hierarchical Indexing

```
import pandas as pd
import numpy as np

data = pd.Series(np.random.randn(6),
index=[['a', 'a', 'b', 'b', 'c', 'c'],
[1, 2, 1, 2, 1, 2]])
print(data)
print(data['b'])
print(data[:, 1])
```

b. Stack and Unstack

```
df = data.unstack()
print(df)
print(df.stack())
```

c. Merge and Combine

```
df1 = pd.DataFrame({'a': [1, np.nan, 3], 'b': [4, 5, 6]})
df2 = pd.DataFrame({'a': [7, 8, 9], 'b': [np.nan, 11, 12]})

merged = df1.combine_first(df2)
print(merged)
```